

HTML5 Video Playback Module

MSE + EME + DRM — explained in simple terms

The project (resume line)

Served **200K+ monthly playback sessions** at a **0.4% error rate** by building an HTML5 video playback module in JavaScript using **MSE** and **EME** with **DRM license handling**, **buffer management**, and **codec detection**.

What is inside this document	Where
1. The project in one page (simple summary)	Chapter 1
2. Words you need to know (glossary)	Chapter 2
3. Architecture diagram — every box and every arrow explained	Chapter 3
4. Every component explained in detail	Chapter 4
5. The complete workflow, phase by phase (with licence and buffer diagrams)	Chapter 5
6. How the numbers (200K+ sessions, 0.4% errors) were achieved and measured	Chapter 6
7. Quick revision / interview sheet	Chapter 7

Prepared for Dev Kumar · October 2026

Note: numbers such as “30-second buffer target” or “3 retries” are typical, realistic example values used to explain the design. Swap in the exact values from your own implementation where they differ.

1. The Project in One Page

1.1 The simple summary

When you press Play on Netflix, Hotstar or Prime Video inside a web browser, a lot happens before you see the first frame. The browser's plain `<video>` tag can only play a simple video file. It cannot, on its own, change quality when your internet slows down, and it cannot play paid, locked (encrypted) movies.

This project was a **JavaScript module that sits between the website and the browser's video engine** and does all of that extra work:

- **It checks what the device can play** (which video formats, which “lock” systems) — this is **codec detection**.
- **It downloads the video in small pieces** (2–6 seconds each) instead of one big file, and feeds them to the browser using **MSE (Media Source Extensions)**.
- **It keeps a healthy “reserve” of video ahead of what you are watching**, removes old pieces to save memory, and avoids pauses — this is **buffer management**.
- **It unlocks protected content**. The video is encrypted; the module asks a licence server for the keys and hands them to the browser's secure decryption box using **EME (Encrypted Media Extensions)** — this is **DRM licence handling**.
- **It watches for problems and fixes them automatically** (retries, switching quality, renewing licences), and reports what happened to an analytics system.

Result in one sentence

The module handled **more than 200,000 viewing sessions every month**, and only **0.4%** of them (about **4 in every 1,000**, roughly 800 a month) ended in an error the viewer could not recover from — **99.6% of sessions played successfully**.

1.2 A simple analogy: a water tank

Think of the video as water flowing to your tap (the screen):

Real-world piece	In the video player
The water company	The CDN server that stores the video pieces
Water arrives in buckets, not one huge pipe	Video is downloaded in small segments (chunks)
A tank on the roof that stores some water	The buffer — video downloaded but not yet watched
Keeping the tank neither empty nor overflowing	Buffer management
Using a smaller bucket when supply is slow	ABR — switching to lower quality on slow internet
A locked tank that needs a key from the company	DRM — encrypted video that needs a licence key
Checking your taps fit the pipe before installing	Codec detection — checking the device can play the format
A meter that records leaks and outages	Telemetry — measuring errors and stalls

1.3 The five pillars of the project

Pillar	What it means	Why it matters
HTML5 video	The browser's built-in <code><video></code> element that shows pictures and plays sound.	No plugins (like old Flash). Works in every modern browser.
MSE	A browser API that lets JavaScript feed video pieces into <code><video></code> one by one.	Enables adaptive quality, seeking, live streams and fine control.
EME + DRM	A browser API that lets JavaScript request keys so the browser's secure module can decrypt locked video.	Studios require DRM before licensing paid content.
Buffer management	Deciding how much video to keep ready, when to download more and when to delete old pieces.	Prevents pauses ("buffering...") and memory crashes.
Codec detection	Checking which compression formats (H.264, HEVC, VP9, AV1, AAC) and DRM systems the device supports.	Avoids picking a format the device cannot decode — a major source of errors.

2. Words You Need to Know

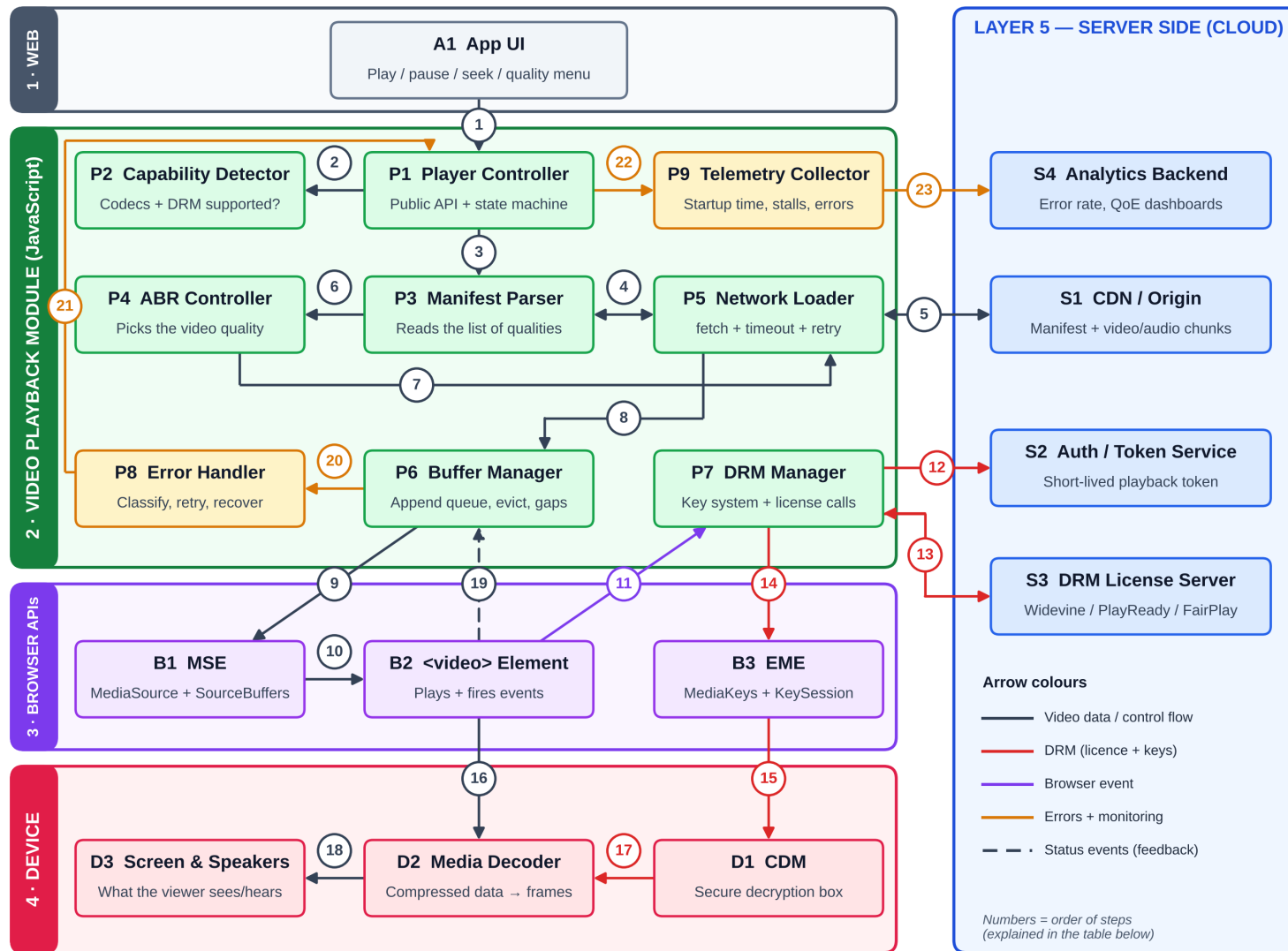
Read this once; every later chapter uses these words.

Term	Simple meaning
Playback session	One viewer watching one video, from pressing Play until they leave or the video ends.
Codec	The method used to squeeze (compress) video or audio. Examples: H.264 (AVC), H.265 (HEVC), VP9, AV1 for video; AAC, Opus, Dolby (EC-3) for audio.
Codec string	A short label describing the exact codec, e.g. <code>avc1.640028</code> = H.264 High profile, level 4.0; <code>mp4a.40.2</code> = AAC-LC audio.
Container	The “box” that holds compressed video/audio, e.g. MP4 (fragmented MP4 / CMAF) or WebM.
Segment (chunk)	A small piece of the video, usually 2–6 seconds long.
Init segment	A tiny header chunk that tells the decoder how to read the following chunks (codec settings, track info, encryption info). Sent once per quality.
Manifest	A text file listing every available quality and where each segment lives. DASH uses <code>.mpd</code> (XML); HLS uses <code>.m3u8</code> playlists.
Rendition / bitrate ladder	The same video encoded at several qualities, e.g. 240p @ 400 kbps up to 1080p @ 6 Mbps.
ABR	Adaptive Bitrate — automatically choosing the best quality for the current internet speed and screen size.
Buffer	Video that has been downloaded and handed to the browser but not yet watched.
Rebuffering / stall	The video freezes because the buffer ran empty (“spinning wheel”).
MSE	Media Source Extensions — browser API (<code>MediaSource</code> , <code>SourceBuffer</code>) that lets JavaScript push chunks into <code><video></code> .
EME	Encrypted Media Extensions — browser API (<code>MediaKeys</code> , <code>MediaKeySession</code>) that connects JavaScript to the device’s decryption module.
DRM	Digital Rights Management — encrypting video so only authorised viewers on approved devices can watch it.
Key system	A specific DRM technology: Google Widevine (<code>com.widevine.alpha</code>), Microsoft PlayReady (<code>com.microsoft.playready</code>), Apple FairPlay (<code>com.apple.fps</code>).
CDM	Content Decryption Module — a sealed, secure part of the browser/OS that holds the keys and decrypts video. JavaScript can never see the keys.
Licence (license)	A message from the licence server containing the decryption keys, locked so only that one CDM can open it.
Licence challenge	The request the CDM creates for the licence server. It is an opaque binary blob — the player just forwards it.
PSSH / initData	Data inside the video (or manifest) saying which key ID is needed and for which DRM system.

Term	Simple meaning
CENC / cbc	Common Encryption — one standard way of encrypting files so the same file works with Widevine, PlayReady and FairPlay (cbc mode).
CDN	Content Delivery Network — many servers around the world that store copies of the video close to viewers.
Telemetry	Data the player sends home about what happened: start time, stalls, errors, quality.
Error rate	Sessions that ended in an unrecoverable (fatal) error ÷ total sessions. Here: 0.4%.

3. Architecture Diagram

Five layers, 20 boxes, 23 numbered arrows. Every box and every arrow is explained on the pages that follow.



3.1 How to read the diagram

- **Boxes** are the parts of the system. Each has a short code: **A** = app, **P** = player module (the code that was written), **B** = browser API, **D** = device, **S** = server.
- **Numbered circles** show the order things happen in, from 1 (user presses Play) to 23 (report sent to analytics).
- **Arrow colours:** dark grey = video data and normal control flow; red = DRM (licence and keys); purple = an event fired by the browser; orange = errors and monitoring; dashed = feedback / status information flowing back.
- A **double-headed arrow** (4, 5, 13) is a request that gets a response back.

3.2 The five layers

Layer	What lives here	Who controls it
1 · Web	The website or app screen: buttons, progress bar, quality menu (A1).	The product / UI team.
2 · Video playback module	The JavaScript code built in this project: nine components P1–P9.	You — this is the project.
3 · Browser APIs	Tools the browser provides: MSE (B1), the <code><video></code> element (B2), EME (B3).	The browser (Chrome, Edge, Safari, Firefox).
4 · Device	Secure decryption (D1 CDM), hardware decoder (D2), screen and speakers (D3).	The operating system and hardware. JavaScript cannot reach inside.
5 · Server side	CDN (S1), token service (S2), DRM licence server (S3), analytics (S4).	Backend / infrastructure teams and DRM vendors.

3.3 Every box in the diagram

Box	Name	What it does (simple)
A1	App UI	Buttons and screens the viewer touches. Calls the player's API (load, play, pause, seek).
P1	Player Controller	The "manager". Offers a simple API, keeps track of the current state and coordinates every other component.
P2	Capability Detector	Asks the browser which codecs, containers and DRM systems it supports before anything is downloaded.
P3	Manifest Parser	Reads the manifest and builds a clean list of qualities, tracks and segment addresses.
P4	ABR Controller	Decides which quality to download next, using measured internet speed, buffer level and screen size.
P5	Network Loader	Downloads manifests and segments with timeouts, retries and speed measurement.
P6	Buffer Manager	Feeds chunks into MSE one at a time, keeps the buffer at the right size, deletes old data and fixes small gaps.
P7	DRM Manager	Chooses the DRM system, sets up EME, requests the licence from the server and handles key status changes.

Box	Name	What it does (simple)
P8	Error Handler & Recovery	Collects errors from all parts, decides whether to retry, switch quality, reset or stop, and shows a friendly message.
P9	Telemetry Collector	Records startup time, stalls, quality changes and errors for each session.
B1	MSE	MediaSource + SourceBuffers : the doorway JavaScript uses to push video chunks into the browser.
B2	<video> element	Plays the media and fires events (playing , waiting , timeupdate , encrypted , error).
B3	EME	MediaKeys + MediaKeySession : the doorway JavaScript uses to talk to the secure CDM.
D1	CDM	Sealed decryption box. Holds keys and decrypts video; keys never leave it.
D2	Media Decoder	Hardware/software that turns compressed data into actual pictures and sound.
D3	Screen & Speakers	Where the viewer finally sees and hears the content.
S1	CDN / Origin	Stores the manifest, init segments and media segments, served from servers close to the viewer.
S2	Auth / Token Service	Confirms the viewer is logged in and allowed to watch; gives a short-lived playback token.
S3	DRM Licence Server	Checks the token and returns a licence (keys) for that one device.
S4	Analytics Backend	Receives telemetry; dashboards show session counts, error rate (0.4%), stalls and quality.

3.4 Every arrow in the diagram

#	From → To	What travels along the arrow and why
1	App UI → Player Controller	player.load(url, drmConfig) then play() . The viewer has asked to watch.
2	Controller → Capability Detector	“Before we start — what can this browser play?” Returns supported codecs and DRM systems.
3	Controller → Manifest Parser	“Load and read this manifest URL.”
4	Manifest Parser ↔ Network Loader	Parser asks the loader to fetch the manifest text; loader returns it.
5	Network Loader ↔ CDN	HTTP GET requests for the manifest, init segments and media segments; the CDN sends the bytes back.
6	Manifest Parser → ABR Controller	The list of playable qualities (already filtered by what the device supports).
7	ABR Controller → Network Loader	“Download the next segment at quality X.” Loader reports back measured speed.
8	Network Loader → Buffer Manager	Downloaded segment bytes (an ArrayBuffer).

#	From → To	What travels along the arrow and why
9	Buffer Manager → MSE	<code>sourceBuffer.appendBuffer(bytes)</code> — one chunk at a time, waiting for <code>updateend</code> .
10	MSE → <code><video></code>	The browser now has playable (still encrypted) media attached to the video element.
11	<code><video></code> → DRM Manager	The <code>encrypted</code> event: “this media is locked; here is the key ID info (<code>initData</code>).”
12	DRM Manager → Auth Service	Gets a short-lived token proving the viewer is entitled to watch.
13	DRM Manager ↔ Licence Server	POST the licence challenge + token; receive the licence containing keys.
14	DRM Manager → EME	Creates the key session, generates the request, then calls <code>session.update(licence)</code> .
15	EME → CDM	The licence is passed into the sealed CDM, which unpacks and stores the keys.
16	<code><video></code> → Media Decoder	Encrypted compressed frames are sent for decoding.
17	CDM → Media Decoder	The CDM decrypts each frame (often inside secure hardware) just before decoding.
18	Decoder → Screen & Speakers	Finished pictures and sound reach the viewer.
19	<code><video></code> → Buffer Manager (dashed)	Status feedback: current time, buffered ranges, <code>waiting/playing</code> events. Drives download and cleanup decisions.
20	Buffer Manager → Error Handler	Any error (append failure, quota full, stall) is reported. Every component reports here, the arrow shows the busiest path.
21	Error Handler → Controller	The decision: retry, switch quality, reset playback, or stop and show a message.
22	Controller → Telemetry	Every state change and error is recorded.
23	Telemetry → Analytics Backend	Session report sent (batched, and with <code>navigator.sendBeacon</code> when the page closes).

4. Every Component Explained

This chapter goes through each box in the diagram. The nine **P** components are the code written in the project, so they get the most detail. The browser, device and server parts are explained afterwards so the full picture is clear.

4.1 Player module components (the code you built)

P1 Player Controller

In simple words	Everyday analogy
The boss of the module. The website only talks to this one object; it tells every other part what to do and keeps track of what state playback is in.	A film director: does not hold the camera, but tells every crew member when to act.

What it does, step by step

- Exposes a small **public API**: `load(url, options)`, `play()`, `pause()`, `seek(seconds)`, `setQuality(level | 'auto')`, `setAudioTrack(lang)`, `destroy()`.
- Runs a **state machine** so the player is always in exactly one clear state: `IDLE` → `LOADING` → `READY` → `PLAYING` ⇌ `PAUSED`, plus `BUFFERING`, `SEEKING`, `ENDED`, `ERROR`. Illegal moves (e.g. seeking before load) are blocked.
- **Orders the start-up**: capability check → manifest → track choice → MSE setup → DRM setup → first segments.
- Acts as an **event emitter** so the UI can react: `statechange`, `qualitychange`, `bufferupdate`, `error`, `ended`.
- **Cleans up** on `destroy()`: cancels downloads, closes DRM sessions, removes listeners, revokes the MediaSource object URL, and resets the `<video>` element so no memory is leaked (important on long-running TV / single-page apps).

Technical details (the actual browser APIs / logic)

- Listens to `<video>` events: `loadedmetadata`, `canplay`, `playing`, `pause`, `waiting`, `seeking`, `seeked`, `ended`, `error`.
- Keeps one **session ID** per playback so every telemetry event can be grouped into one session.

What can go wrong — and how it is handled

- `video.play()` can be **rejected by autoplay rules** (browser blocks sound without a user click) → controller catches the promise rejection and shows a Play button or starts muted, instead of failing.
- Fast clicking (play/pause/seek spam) can create races → the state machine and an operation queue make actions run in order.

P2 Codec & Capability Detector

In simple words	Everyday analogy
Before downloading anything, it asks the browser: “Which video formats and which DRM systems can you actually play here?” The answer filters the manifest so the player never picks something impossible.	Checking that your charger fits the socket before you travel.

What it does, step by step

- Checks that **MSE exists** (`window.MediaSource`, or `ManagedMediaSource` on newer iPhones/iPads).
- Tests each **codec string** from the manifest, e.g. `video/mp4; codecs="avc1.640028", hvc1.1.6.L120.90, vp09.00.40.08, av01.0.08M.08, mp4a.40.2, ec-3`.
- Asks whether playback will be **smooth and power-efficient** at a given resolution / frame rate (not just “supported”).
- Tests **DRM key systems** in a preferred order (e.g. Widevine → PlayReady → FairPlay) and the **robustness level** (hardware-secure vs software).
- Returns a **capabilities object** that the Manifest Parser and ABR use to remove unsupported qualities (e.g. no HEVC on a browser without it, no 4K if only software DRM is available).
- Results are **cached** for the session (these checks can take tens of milliseconds).

Technical details (the actual browser APIs / logic)

- `MediaSource.isTypeSupported(mime)` → true/false for MSE.
- `video.canPlayType(mime)` → "", "maybe" or "probably".
- `navigator.mediaCapabilities.decodingInfo({ type: 'media-source', video: { contentType, width, height, bitrate, framerate } })` → { supported, smooth, powerEfficient }.
- `navigator.requestMediaKeySystemAccess(keySystem, [config])` with `videoCapabilities` / `audioCapabilities` and robustness such as `HW_SECURE_ALL` or `SW_SECURE_CRYPT0`.

What can go wrong — and how it is handled

- Some browsers say “supported” but play badly → prefer `mediaCapabilities` (gives `smooth`), keep a small **deny-list** of known bad device/codec combos.
- No supported codec or DRM at all → a clear “**device not supported**” message instead of a confusing failure later. This early check removes a whole class of errors.

```

async function detect(codecs, keySystems) {
  const ok = codecs.filter(c => MediaSource.isTypeSupported(c));
  for (const ks of keySystems) {
    try {
      const access = await navigator
        .requestMediaKeySystemAccess(ks.name, [ks.config]);
      return { codecs: ok, keySystem: ks.name, access };
    } catch (e) { /* not supported, try next */ }
  }
  return { codecs: ok, keySystem: null };
}

```

P3 Manifest Parser

In simple words	Everyday analogy
Reads the manifest (the video's "menu") and turns it into a simple, uniform list: the qualities available, the audio languages, subtitles, and the address of every segment.	Reading a restaurant menu and noting only the dishes you are allowed to eat.

What it does, step by step

- Downloads the manifest through the Network Loader.
- **DASH (.mpd, XML)**: walks `Period` → `AdaptationSet` (`video/audio/text`) → `Representation` and reads `bandwidth`, `width`, `height`, `codecs`, `frameRate`; expands `SegmentTemplate` (`$Number$ / $Time$`) or `SegmentTimeline` into real segment URLs and times.
- **HLS (.m3u8)**: reads the master playlist (`#EXT-X-STREAM-INF` with `BANDWIDTH`, `RESOLUTION`, `CODECS`), then media playlists (`#EXTINF` durations, `#EXT-X-MAP` for the init segment, `#EXT-X-KEY` for encryption).
- Reads **DRM information**: `ContentProtection` elements / PSSH boxes and the **key IDs** so the DRM Manager can prepare early.
- Builds an internal model: `tracks[]` → `renditions[]` → `segments[{ url, start, duration, byteRange }]`.
- **Filters** renditions using the Capability Detector's results.
- For **live** streams it re-downloads the manifest periodically to learn about new segments.

What can go wrong — and how it is handled

- Malformed or unexpected manifest → validation with clear error codes (e.g. `MANIFEST_PARSE_ERROR`) instead of a crash.
- After filtering, **no playable rendition remains** → reported as a capability error with details for debugging.

P4 ABR Controller (Adaptive Bitrate)

In simple words	Everyday analogy
Chooses which quality to download next. Fast internet → sharper video; slow internet → lower quality so the video keeps playing instead of freezing.	Changing gears in a car depending on the slope of the road.

What it does, step by step

- **Measures throughput** from every download (`bytes ÷ time`) and smooths it with two moving averages — a fast one and a slow one — then uses the **lower** of the two to stay safe.
- Applies a **safety factor** (e.g. use only ~80% of measured speed) so a small dip does not cause a stall.
- Looks at the **buffer level**: lots of buffer → can afford to try higher quality; low buffer → go down quickly.
- **Caps quality to the player size**: no 1080p in a 360-pixel-wide player — saves bandwidth for the viewer and the company.
- **Switches down fast, up slowly** (needs a few good measurements) to avoid flickering between qualities.

- Checks **dropped frames** — if the device cannot decode a quality smoothly, that quality is avoided.
- Respects a **manual choice** if the viewer picks a quality from the menu.
- Chooses a sensible **starting quality** (often a middle rung, or based on the last session's speed) so the first frame appears fast.

Technical details (the actual browser APIs / logic)

- Speed samples come from the Network Loader; buffer level from `sourceBuffer.buffered` / `video.buffered`.
- Dropped frames from `video.getVideoPlaybackQuality()` (`droppedVideoFrames ÷ totalVideoFrames`).
- Player size from `video.clientWidth/clientHeight × devicePixelRatio`.

What can go wrong — and how it is handled

- Sudden network drop mid-download → the loader can **abort** a slow high-quality download and fetch the same segment at lower quality before the buffer runs out.

P5 Network Loader

In simple words	Everyday analogy
The delivery person. Downloads every file the player needs (manifest, init segments, media segments, sometimes licence calls) and makes downloads reliable.	A courier who retries if nobody answers the door and reports how long each delivery took.

What it does, step by step

- Uses `fetch()` and returns the response as an `ArrayBuffer`.
- Adds a **timeout** to every request with `AbortController`, so a hung connection does not freeze playback.
- **Retries** temporary failures (timeouts, HTTP 5xx, network drops) with **exponential backoff plus jitter** (e.g. wait 0.5 s, 1 s, 2 s, with a small random extra so thousands of viewers don't retry at the same instant).
- Does **not** blindly retry permanent failures (HTTP 403/404) — those are passed to the Error Handler (e.g. a 403 may mean an expired signed URL → refresh the token).
- Can switch to a **backup CDN** address if one CDN keeps failing.
- Supports **byte-range requests** (`Range: bytes=...`) when segments are parts of one big file.
- Records **bytes and time** for every download and sends them to the ABR Controller.
- Can **cancel** downloads that are no longer needed (after a seek or quality switch).

What can go wrong — and how it is handled

- Flaky mobile networks are the biggest single source of errors → timeouts + retries + CDN failover turn most of them into short delays instead of failed sessions.

P6 Buffer Manager

In simple words	Everyday analogy
Keeps the right amount of video ready ahead of the viewer, pushes chunks into the browser in the correct order, deletes old video to save memory, and fixes small holes that would cause freezes.	Keeping a water tank neither empty (video freezes) nor overflowing (browser runs out of memory).

What it does, step by step

- **Separate queues for video and audio** — each has its own `SourceBuffer`.
- **One append at a time:** a `SourceBuffer` cannot accept a new chunk while `updating` is true, so chunks wait in a queue; the next one is appended when the `updateend` event fires.
- **Forward buffer goal:** keep e.g. ~30 seconds downloaded ahead of the playhead. Below the goal → ask for more; at the goal → pause downloading (saves data).
- **Low-water mark:** if the buffer drops below e.g. ~5 seconds, tell ABR to lower quality urgently.
- **Back buffer & eviction:** keep only e.g. ~30 seconds behind the playhead (for quick rewind) and delete the rest with `sourceBuffer.remove(start, end)`.
- **Gap jumping:** sometimes two chunks leave a tiny hole (a few milliseconds) in `buffered` ranges; the video would freeze there forever. The manager detects small gaps and nudges `currentTime` just past them.
- **Stall detection:** if the video is “playing” but `currentTime` has not moved for a while, it reports a stall and tries a nudge or recovery.
- **Quality switch:** appends the new rendition’s **init segment** first, then its media segments.
- **Seek:** cancels pending downloads, clears the queue, aborts the current append, and starts loading from the segment that contains the new time.
- **End of video:** when the last segment is appended, calls `mediaSource.endOfStream()` so the browser knows playback can finish.

Technical details (the actual browser APIs / logic)

- `mediaSource.addSourceBuffer('video/mp4; codecs="avc1.640028"')`, `appendBuffer()`, `remove()`, `abort()`, `buffered` (`TimeRanges`), `timestampOffset`, `mode`.
- Events: `updateend`, `error` on the `SourceBuffer`; `sourceopen`, `sourceended` on the `MediaSource`.

What can go wrong — and how it is handled

- **QuotaExceededError** (browser memory for media is full, common on TVs and phones) → evict more back buffer, lower the buffer goal, and retry the same append.
- **InvalidStateError** (appending while busy or after the `MediaSource` closed) → prevented by the queue and by checking `readyState === 'open'`.
- Decode error after an append (corrupt chunk) → skip/refetch the segment or switch rendition.

```
function enqueue(sb, queue, bytes) {
  queue.push(bytes);
  if (!sb.updating) appendNext(sb, queue);
}
function appendNext(sb, queue) {
  if (!queue.length || sb.updating) return;
  try { sb.appendBuffer(queue[0]); queue.shift(); }
  catch (e) {
    if (e.name === 'QuotaExceededError') evictAndRetry(sb, queue);
    else reportError(e);
  }
}
sb.addEventListener('updateend', () => appendNext(sb, queue));
```

P7 DRM Manager

In simple words	Everyday analogy
Unlocks protected video. It notices the video is encrypted, asks the licence server for the keys (with proof that the viewer is allowed), and gives the keys to the browser's secure decryption box.	A hotel receptionist: checks your booking (token), asks the key office for a room key (licence), and hands the key straight to the door lock (CDM) — you never see how the key is cut.

What it does, step by step

- **Chooses the key system** that the device supports (from P2): Widevine on Chrome/Firefox/Android, PlayReady on Edge/Windows/Xbox/many TVs, FairPlay on Safari/iOS.
- **Creates and attaches MediaKeys** to the `<video>` element early, before the first encrypted chunk arrives (avoids a delay).
- For FairPlay (and optionally Widevine), loads the **server certificate** with `setServerCertificate()`.
- Listens for the **encrypted** event (or uses PSSH data from the manifest) to learn which key is needed.
- Creates a **key session** and calls `generateRequest(initDataType, initData)`.
- Receives the **message** event containing the licence challenge and **POSTs it to the licence server** with the viewer's token (headers, retry, timeout).
- Calls `session.update(licence)` with the response so the CDM can load the keys.
- Watches **keystatuschanges**: **usable** (all good), **expired** (renew), **output-restricted** (screen connection not secure enough, e.g. no HDCP → drop to a lower resolution allowed without it), **internal-error**.
- Handles **licence renewal** for long sessions (`message` with type `license-renewal`).
- **De-duplicates** requests (one session per key ID) and **closes sessions** on `destroy()`.

Technical details (the actual browser APIs / logic)

- `navigator.requestMediaKeySystemAccess()`, `access.createMediaKeys()`, `video.setMediaKeys()`, `mediaKeys.createSession('temporary')`, `session.generateRequest()`, `session.update()`, `session.close()`, `session.keyStatuses`.
- Content is usually encrypted once with **CENC cbcs** so the same files work for all three DRM systems.

What can go wrong — and how it is handled

- Licence server **timeout / 5xx** → retry with backoff.
- **401/403** (token expired or not entitled) → refresh the token once, then show “not authorised” rather than a generic error.
- **output-restricted** → automatically remove resolutions that need HDCP and continue at a lower quality instead of failing.
- Device lacks hardware-secure DRM → play only the qualities allowed for software DRM (studios often allow up to 480p/720p).

P8 Error Handler & Recovery

In simple words	Everyday analogy
Every problem in the player goes to this one place. It decides: is this temporary (try again), fixable (change something), or final (stop nicely and explain)? This component is the main reason the error rate stayed at 0.4%.	A hospital triage nurse: minor issues are fixed on the spot, serious ones get special treatment, and only the truly critical are escalated.

What it does, step by step

- **Classifies** every error: category (network, manifest, media/decode, MSE, DRM, capability), **retryable or not**, **fatal or not**.
- Gives each error a **stable error code** (e.g. `NET_TIMEOUT`, `DRM_LICENSE_403`, `MEDIA_DECODE`) so dashboards can count and compare them.
- Runs a **recovery ladder**, trying the cheapest fix first:
 - 1. Retry the same request.
 - 2. Switch to another quality or another CDN.
 - 3. Re-request the licence / refresh the token.
 - 4. **Reset playback**: tear down and recreate the MediaSource at the same position (fixes many stuck-decoder problems).
 - 5. Only then mark the session **fatal** and show a friendly message with a retry button.
- Limits retries (a **retry budget**) so the player never loops forever.
- Sends every error — recovered or fatal — to Telemetry with context (browser, device, codec, rendition, position).

Technical details (the actual browser APIs / logic)

- `video.error.code`: `MEDIA_ERR_ABORTED` (1), `MEDIA_ERR_NETWORK` (2), `MEDIA_ERR_DECODE` (3), `MEDIA_ERR_SRC_NOT_SUPPORTED` (4).
- Also catches SourceBuffer **error** events, rejected promises (EME, `play()`), and fetch failures.

P9 Telemetry Collector

In simple words	Everyday analogy
A flight recorder for every viewing session. It notes when playback started, every stall, every quality change and every error, and sends a summary to the analytics system. This is how the 200K+ sessions and the 0.4% error rate were measured.	A fitness tracker that records your activity and syncs it to an app.

What it does, step by step

- Creates a **session record** with a unique ID when `load()` is called.
- Records key **events**: `session_start`, `first_frame`, `stall_start` / `stall_end`, `quality_change`, `seek`, `error` (code, fatal or recovered), `session_end`.
- Computes **Quality of Experience (QoE) metrics**: video start time (time to first frame), video start failures, exits before video start, rebuffering ratio (stall time ÷ watch time), average bitrate, dropped frames.
- Sends data in **batches** to save requests, and uses `navigator.sendBeacon()` on `pagehide` so the last report is not lost when the tab closes.
- Never sends personal data or keys — only technical information.

4.2 Browser APIs (provided by the browser)

B1 MSE — Media Source Extensions

In simple words	Everyday analogy
A doorway that lets JavaScript hand video to the <code><video></code> element piece by piece instead of giving it one file.	A conveyor belt into a factory: you place boxes on the belt one at a time.

What it does, step by step

- `const ms = new MediaSource(); video.src = URL.createObjectURL(ms);` connects it to the video element.
- Wait for `sourceopen`, then `ms.addSourceBuffer(mimeWithCodecs)` — one for video, one for audio.
- Chunks are pushed with `appendBuffer`; `ms.duration` sets the length; `ms.endOfStream()` marks the end.

B2 The `<video>` element

In simple words	Everyday analogy
The visible player window. It decodes and shows the media and tells everyone what is happening through events.	The TV screen and its status lights.

What it does, step by step

- Properties used constantly: `currentTime`, `buffered`, `paused`, `readyState`, `playbackRate`, `error`.

- Events listened to: `loadedmetadata`, `canplay`, `playing`, `waiting`, `timeupdate`, `seeking`, `seeked`, `ended`, `encrypted`, `error`.

B3 EME — Encrypted Media Extensions

In simple words	Everyday analogy
A doorway between JavaScript and the secure CDM. JavaScript can pass messages through it, but it can never see the keys.	A bank's deposit slot: you can push an envelope in, but you can't reach inside the vault.

What it does, step by step

- `MediaKeySystemAccess` → `MediaKeys` (attached to the video) → `MediaKeySession` (one per licence).
- Fires `message` (challenge ready) and `keystatuseschange` (keys ready / expired / restricted).

4.3 Device layer (hardware / secure zone)

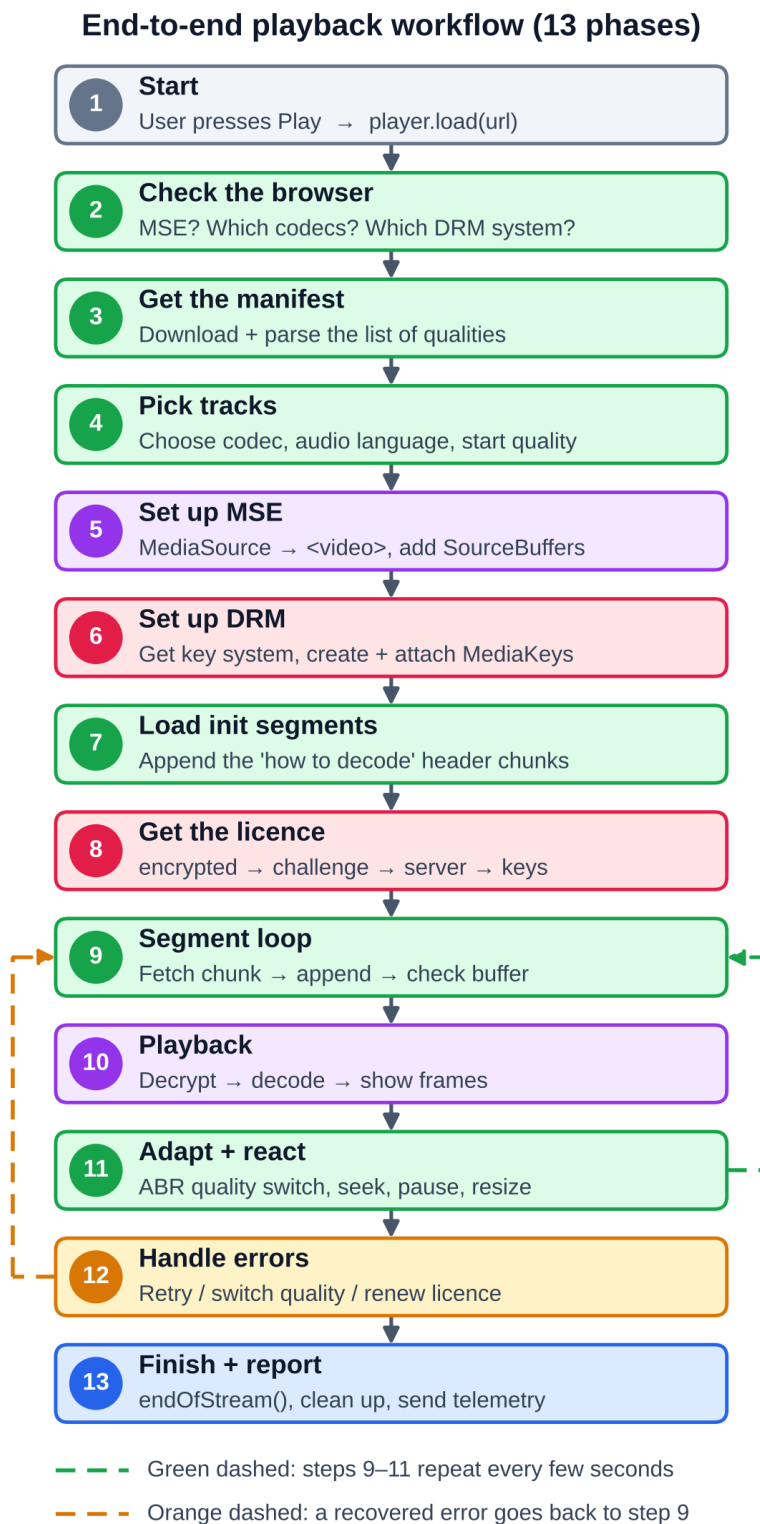
Box	What it is	Important detail
D1 · CDM	Content Decryption Module — a sealed component built into the browser or operating system by the DRM vendor.	Security levels differ: e.g. Widevine L1 decrypts inside secure hardware (allowed HD/4K), L3 is software-only (often limited to SD). The player must respect these limits.
D2 · Media decoder	Hardware chip or software that turns compressed video into frames.	Hardware decoding saves battery. Not every device decodes every codec — hence codec detection.
D3 · Screen & speakers	Final output.	For HD/4K, the connection to an external screen may need HDCP copy protection; otherwise EME reports <code>output-restricted</code> .

4.4 Server side (cloud)

Box	What it is	Important detail
S1 · CDN / Origin	Stores and delivers manifests and segments from servers near the viewer.	URLs are often signed (expire after some time). Multiple CDNs give failover.
S2 · Auth / Token service	Confirms the viewer's login and subscription.	Issues a short-lived token (e.g. a JWT) that the licence request carries.
S3 · DRM licence server	Usually the company's licence proxy in front of the DRM vendor's service.	Checks the token/entitlement, adds rules (resolution limits, expiry, offline or not) and returns the licence.
S4 · Analytics backend	Collects telemetry from all players.	Dashboards show monthly sessions (200K+), error rate (0.4%), start time, rebuffering, broken down by browser, device, CDN and error code.

5. The Complete Workflow, End to End

This chapter follows one viewing session from the moment the viewer presses Play until the session report is sent. The diagram gives the big picture; each phase is then explained in detail.



Phase 1 — Start: the viewer presses Play

In simple words: The website hands the video address and DRM settings to the player and says “start”.

What exactly happens

- UI calls `player.load(manifestUrl, { drm: { licenseUrl, tokenProvider } })`.
- Controller moves from **IDLE** to **LOADING**, creates a new **session ID**, and Telemetry records `session_start` with a timestamp (the start-time clock begins).
- If something was already playing, the old session is fully cleaned up first (downloads cancelled, DRM sessions closed).

What can go wrong — and how it is handled

- `load()` called twice quickly → the older request is cancelled so only the newest video loads.

Phase 2 — Check the browser (codec detection)

In simple words: The player asks the device what it can play, so it never chooses something impossible.

What exactly happens

- Check MSE support (`MediaSource` / `ManagedMediaSource`).
- Check DRM: try `requestMediaKeySystemAccess` for Widevine, PlayReady and FairPlay in order; keep the first that works and note its robustness (hardware or software).
- Check codecs: `MediaSource.isTypeSupported` and `mediaCapabilities.decodingInfo` for each codec in the manifest (done after the manifest arrives, or with a known list).
- Store the results in a capabilities object for this session.

What can go wrong — and how it is handled

- No MSE or no DRM → stop immediately with “This browser/device is not supported” and a hint (e.g. update the browser).

Phase 3 — Get the manifest

In simple words: Download the “menu” of the video and turn it into a list the player understands.

What exactly happens

- Network Loader fetches the `.mpd` or `.m3u8` with timeout and retry.
- Parser builds tracks → renditions → segment lists, reads codecs, bitrates, resolutions, duration and DRM key IDs.
- Unsupported renditions are removed using Phase 2’s results.

What can go wrong — and how it is handled

- Network failure → retried. Invalid manifest → clear `MANIFEST_*` error code. Nothing playable after filtering → capability error.

Phase 4 — Pick tracks and the starting quality

In simple words: Choose which video version, which audio language and which quality to start with.

What exactly happens

- Pick the **best codec family** the device supports (e.g. HEVC or AV1 for smaller files if supported smoothly, otherwise H.264).
- Pick the **audio track** from the viewer's language preference (fallback to the default).
- ABR picks a **starting quality** — usually a middle level, or based on speed from the previous session — so the first frame appears fast without stalling.
- Remove qualities the **DRM level** does not allow (e.g. HD on software-only DRM) and qualities above the **player size**.

Phase 5 — Set up MSE

In simple words: Open the “conveyor belt” between the player code and the `<video>` element.

What exactly happens

- Create new `MediaSource()` and attach it: `video.src = URL.createObjectURL(mediaSource)`.
- Wait for the `sourceopen` event.
- Set `mediaSource.duration` (for on-demand video).
- Create one `SourceBuffer` per track type with the exact codec string, e.g. `addSourceBuffer('video/mp4; codecs="avc1.640028"')` and `addSourceBuffer('audio/mp4; codecs="mp4a.40.2"')`.

What can go wrong — and how it is handled

- `addSourceBuffer` throws `NotSupportedError` → fall back to another codec family found in Phase 2, rather than failing.

```
const ms = new MediaSource();
video.src = URL.createObjectURL(ms);
ms.addEventListener('sourceopen', () => {
  ms.duration = manifest.duration;
  videoSB = ms.addSourceBuffer('video/mp4; codecs="avc1.640028"');
  audioSB = ms.addSourceBuffer('audio/mp4; codecs="mp4a.40.2"');
  startLoading();
}, { once: true });
```

Phase 6 — Set up DRM (before any encrypted data arrives)

In simple words: Prepare the lock-and-key system early so unlocking is quick later.

What exactly happens

- Using the access object from Phase 2: `mediaKeys = await access.createMediaKeys()`.
- Attach it: `await video.setMediaKeys(mediaKeys)`.
- For FairPlay (optionally Widevine): download the **server certificate** and call `mediaKeys.setServerCertificate(cert)`.

- If the manifest already contains PSSH/key IDs, the licence request can start **right away in parallel** with the first segment downloads — this cuts start-up time.

What can go wrong — and how it is handled

- `setMediaKeys` fails → retry once, then report `DRM_SETUP_FAILED` with the key system name for debugging.

Phase 7 — Load the init segments

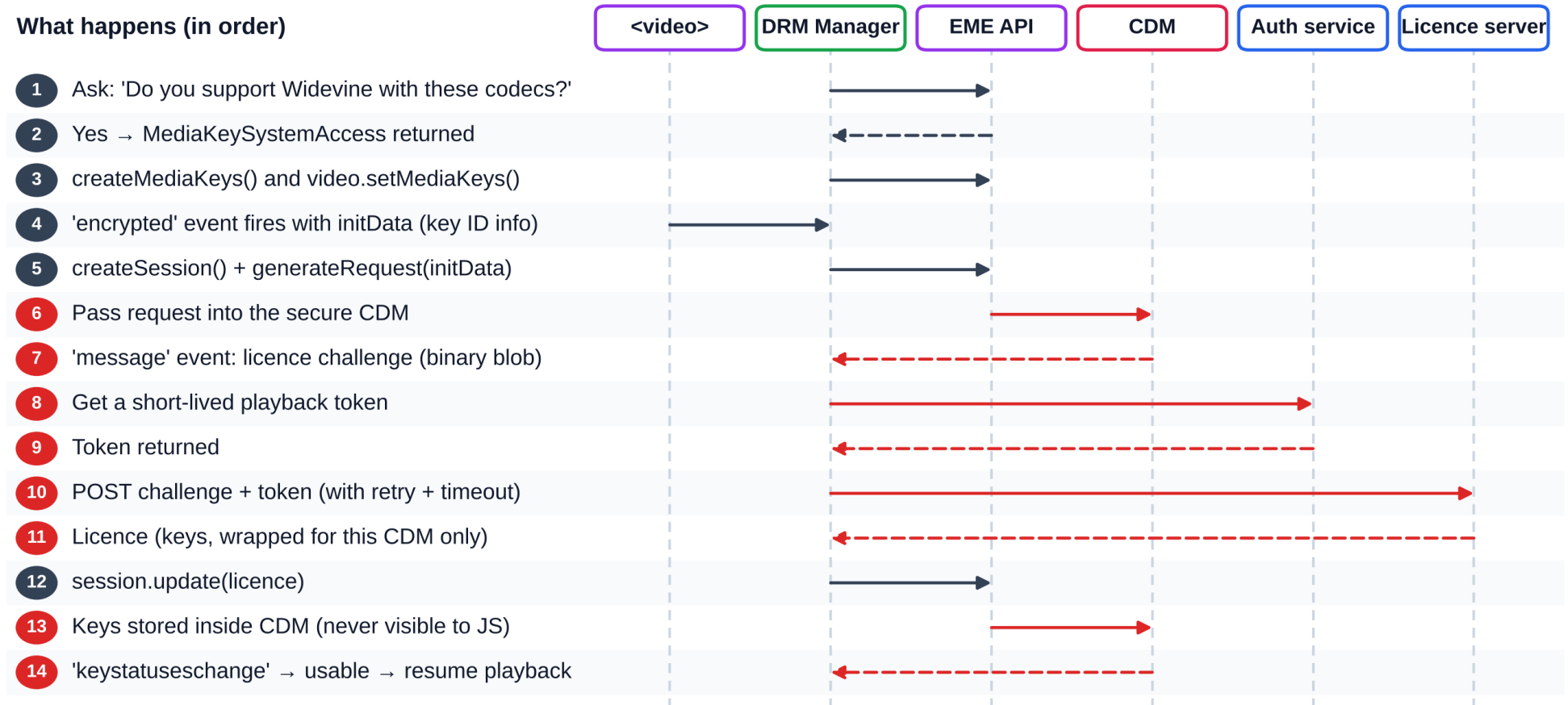
In simple words: Send the small header chunks that tell the decoder how to read everything that follows.

What exactly happens

- Fetch the init segment for the chosen video and audio renditions.
- Append each to its SourceBuffer through the Buffer Manager's queue.
- The browser now knows codec details, resolution and the **encryption info** (the `pssh/tenc` boxes). Because the content is encrypted, the `<video>` element fires the **encrypted** event — which starts Phase 8.

Phase 8 diagram — the licence (DRM) flow in 14 steps

Read top to bottom. Red rows involve the secure CDM or the servers; dashed arrows are replies. JavaScript only carries messages — it never sees the keys.



Phase 8 — Get the licence (DRM licence handling)

In simple words: The video is locked. The player gets the key from the licence server and puts it into the browser's secure box. This follows the 14 steps in the diagram on the previous page.

What exactly happens

- **Steps 1–3 (already done in Phase 6):** key system chosen, MediaKeys created and attached.
- **Step 4:** `<video>` fires `encrypted` with `initDataType` (usually `cenc`) and `initData` (the PSSH: which key is needed).
- **Step 5:** DRM Manager creates a session — `mediaKeys.createSession('temporary')` — and calls `session.generateRequest(initDataType, initData)`. Duplicate key IDs are ignored so only one request is made per key.
- **Steps 6–7:** the CDM builds a **licence challenge** (an opaque binary blob) and the session fires a `message` event with it.
- **Steps 8–9:** the player gets a fresh **playback token** from the auth service (or uses a cached one that has not expired).
- **Steps 10–11:** the player **POSTs** the challenge to the licence server with the token in a header, with timeout and retry. The server checks the viewer's rights and returns the **licence**, encrypted so only this CDM can open it.
- **Steps 12–13:** `await session.update(licence)` — the CDM unpacks the keys and stores them internally.
- **Step 14:** `keystatuschange` fires; when the key status is `usable`, decryption can begin and playback continues.

What can go wrong — and how it is handled

- Licence server slow or down → timeout + retry with backoff; if it keeps failing, `DRM_LICENSE_UNAVAILABLE`.
- Token expired (401/403) → refresh token once, then retry; if still refused → “You don't have access to this title.”
- Key status `output-restricted` → remove HD renditions that need HDCP and continue in lower quality.
- Key status `expired` during long viewing → renewal request through the same session.

```
video.addEventListener('encrypted', async (e) => {
  const session = mediaKeys.createSession('temporary');
  session.addEventListener('message', async (m) => {
    const token = await getToken();
    const res = await fetchWithRetry(licenseUrl, {
      method: 'POST', body: m.message,
      headers: { Authorization: `Bearer ${token}` } });
    await session.update(await res.arrayBuffer());
  });
  await session.generateRequest(e.initDataType, e.initData);
});
```

Phase 9 — The segment loop (buffer management in action)

In simple words: A repeating cycle: download the next chunk, put it into the buffer, check the buffer level, repeat — like topping up a water tank.

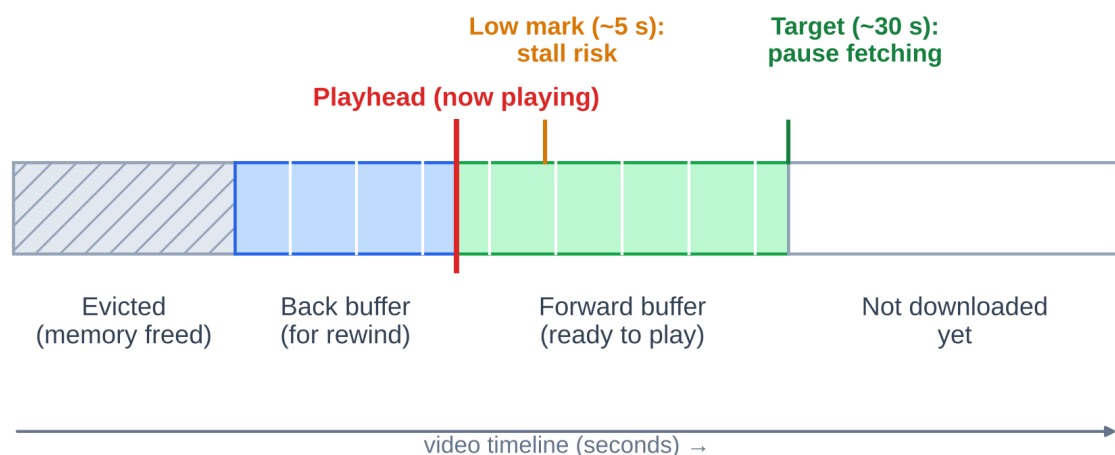
What exactly happens

- Runs on a timer (e.g. every few hundred milliseconds) and after each append.
- **Measure** forward buffer = end of the buffered range containing `currentTime` - `currentTime`.
- If below the **goal** (e.g. ~30 s): ask ABR for the quality → Network Loader fetches the next segment → Buffer Manager queues and appends it.
- If at or above the goal: **wait** (no wasted data if the viewer leaves early).
- **Evict** old data: remove anything more than ~30 s behind the playhead.
- **Check for gaps** ahead of the playhead and plan to jump small ones.
- When the **last segment** is appended: `mediaSource.endOfStream()`.
- **Live streams:** the loop also refreshes the manifest and stays a safe distance behind the live edge.

What can go wrong — and how it is handled

- `QuotaExceededError` → evict more, lower the goal, retry the append.
- Download too slow for current quality → abort and refetch at a lower quality before the buffer empties.

What the buffer looks like at any moment



Left to right: data already deleted to free memory (grey), data kept for quick rewinding (blue), the playhead (red line), data ready to play (green), and data not downloaded yet (white). The aim is to keep the green part between the low mark and the target.

Phase 10 — Playback: decrypt → decode → show

In simple words: The browser and device do the heavy work; the player watches and measures.

What exactly happens

- `video.play()` is called once enough buffer exists (`canplay`). Telemetry records `first_frame` → this gives the **video start time**.
- For each frame: the encrypted data goes from the `<video>` pipeline to the **CDM**, which decrypts it (often inside secure hardware), then the **decoder** produces pictures and sound for the screen and speakers.
- Audio and video stay in sync automatically using the timestamps inside the segments.

What can go wrong — and how it is handled

- Autoplay blocked → show a Play button / start muted. Decode error → handled in Phase 12.

Phase 11 — Adapt and react while watching

In simple words: The player keeps adjusting to the network and to the viewer's actions.

What exactly happens

- **ABR switch:** network speed changed → next segments come from a different quality; the new **init segment** is appended first, so the change is seamless.
- **Seek:** cancel downloads, clear queues, find the segment containing the new time, refill the buffer, resume. The state goes `SEEKING` → `BUFFERING` → `PLAYING`.
- **Stall (waiting event):** state `BUFFERING`, stall timer starts (for telemetry), ABR lowers quality; when `playing` fires the stall ends.
- **Pause / tab hidden:** downloading can slow down or stop to save data.
- **Player resized / fullscreen:** ABR's size cap updates, allowing higher (or lower) quality.

Phase 12 — Handle errors and recover

In simple words: Something went wrong — fix it quietly if possible, otherwise stop nicely and explain.

What exactly happens

- Error reaches the Error Handler with a code and context.
- Recovery ladder: **retry** → **switch quality / CDN** → **refresh licence or token** → **reset playback** (new MediaSource at the same position) → **fatal**.
- If recovered, the session continues at the segment loop (Phase 9); the error is still logged as `*recovered*`.
- If fatal: playback stops, a friendly message with a **Retry** button is shown, and the session is counted in the **error rate**.

Phase 13 — Finish and report

In simple words: The video ends or the viewer leaves; the player cleans up and sends the final report.

What exactly happens

- On the last segment: `mediaSource.endOfStream()` → the `<video>` element fires `ended`.
- On `destroy()` / leaving the page: abort downloads, `session.close()` for every DRM session, remove listeners, `URL.revokeObjectURL()`, clear the video source.
- Telemetry sends `session_end` with totals (watch time, stall count and time, average bitrate, errors, fatal yes/no) using `navigator.sendBeacon()` so it is delivered even while the tab closes.
- The analytics backend counts this as **one of the 200K+ monthly sessions**, and as an error session only if it ended fatally.

6. How the Numbers Were Achieved and Measured

6.1 What the numbers mean

Number	Meaning in simple terms
200K+ monthly playback sessions	More than 200,000 separate “someone pressed Play” events every month were handled by this module — real production traffic, many browsers and devices.
0.4% error rate	Out of every 1,000 sessions, about 4 ended in an error the player could not recover from. With 200,000 sessions that is roughly 800 per month; 199,200 played fine (99.6%).

How the error rate is calculated

Error rate = sessions that ended with a fatal error ÷ total sessions × 100.

Example: 800 fatal sessions ÷ 200,000 sessions × 100 = **0.4%**.

Errors that were **recovered** automatically (a retry that worked, a quality switch) are logged but do **not** count as failed sessions — the viewer kept watching.

6.2 What kept the error rate low

Common cause of failure	What the module did about it	Component
Device cannot play the chosen format	Codec + DRM detection before choosing; filter unsupported renditions; deny-list bad device/codec combos.	P2, P3
Flaky or slow networks	Timeouts, retries with backoff and jitter, CDN failover, ABR stepping down quickly.	P5, P4
Buffer running empty (stalls)	Buffer goal + low-water mark, quality drop when low, abort-and-refetch at lower quality.	P6, P4
Browser media memory full	Back-buffer eviction, smaller goal and retry on QuotaExceededError .	P6
Tiny gaps between chunks causing a permanent freeze	Gap detection and jumping; stall watchdog.	P6
Licence server hiccups / expired tokens	Retry with backoff, token refresh, licence renewal.	P7
Screen connection not secure enough (HDCP)	React to output-restricted by dropping to allowed resolutions instead of failing.	P7
Stuck or crashed decoder	Reset playback: new MediaSource at the same position.	P8
Unknown new problems	Every error has a code + context in telemetry, so the top causes could be found and fixed release by release.	P8, P9

6.3 Other quality metrics usually tracked alongside

Video start time (Play → first frame), **video start failures** (never showed a frame), **exits before video start** (viewer gave up waiting), **rebuffering ratio** (stall time ÷ watch time) and **average bitrate** (how sharp the picture was).

7. Quick Revision / Interview Sheet

7.1 The 30-second explanation

“I built a JavaScript video playback module for the browser. It checks what codecs and DRM the device supports, parses DASH/HLS manifests, and streams video in small segments into the `<video>` element through Media Source Extensions. A buffer manager keeps about 30 seconds ready, evicts old data and fixes gaps, while an ABR controller picks quality from bandwidth and buffer level. For protected content I used Encrypted Media Extensions to request licences from our licence server for Widevine, PlayReady or FairPlay. A central error handler retries, switches quality or resets playback before failing, and telemetry tracked every session — we served over 200,000 sessions a month at a 0.4% error rate.”

7.2 Likely questions and short answers

Question	Short answer
Why not just set <code>video.src</code> to an MP4 file?	No quality switching, no fine buffer control, poor seeking on long videos, and no standard way to do multi-DRM streaming. MSE gives the player that control.
Why do you need an append queue?	A SourceBuffer can only process one append at a time (<code>updating</code> is true until <code>updateend</code>). Appending while busy throws <code>InvalidStateError</code> .
What is <code>QuotaExceededError</code> and how did you handle it?	The browser's media memory is full. Evict back buffer with <code>remove()</code> , reduce the buffer goal, and retry the same chunk.
What is the difference between MSE and EME?	MSE moves data into the video element; EME moves keys (via licences) into the secure CDM. They work together for protected streaming.
Can JavaScript read the decryption keys?	No. Licences are encrypted for the CDM; JavaScript just forwards opaque messages. That is what makes DRM trustworthy.
How do you support three DRM systems?	Encrypt once with CENC <code>cbs</code> , include PSSH for each system, detect which key system the browser supports, and use the matching licence URL/certificate.
How did you detect codecs?	<code>MediaSource.isTypeSupported</code> , <code>canPlayType</code> , and <code>mediaCapabilities.decodingInfo</code> (which also reports smooth/power-efficient), plus <code>requestMediaKeySystemAccess</code> for DRM.
How does ABR avoid flickering?	Smoothed bandwidth (fast + slow averages, take the lower), safety factor, switch down fast and up slowly, and use buffer level as a second signal.
What happens when a seek happens?	Abort downloads and the current append, clear queues, fetch from the segment containing the new time, then resume.
How did you measure the 0.4%?	Telemetry per session with a fatal/non-fatal flag; error rate = fatal sessions ÷ all sessions, viewed in analytics dashboards by browser, device and error code.